Data Pipeline and Feature Engineering

A Surrogate ML Model for a Ti-6Al-4V Cementless Hip Implant

Ata Bulut
Computer Engineering · 220901582

May 2026

ENS003 — Digital Twins for Health Sciences · Instructor: Prof. Dr. Șenol Pișkin

In this talk

- Turning the FEA team's raw output into something an ML model can eat
- The pipeline from ANSYS export to `data/dataset.csv`
- Architecture of `convert.py` and `preprocess.py`
- Bugs I hit, tests and coverage

My responsibility (software team)

Data conversion, schema design, feature engineering, and the CV grouping helper.

The data source

ANSYS Workbench parametric Design Point sweep:

- $m \in \{50, 60, 70, 80, 90, 100\}\,\mathrm{kg}$ (6 values)
- $K \in \{2, 2.5, \ldots, 6\}$ (9 values, body-weight multiplier)
- $\theta \in \{$0ř, 10ř, 20ř$\}$ (3 values, force tilt angle)

$$\mathbf{6} \times \mathbf{9} \times \mathbf{3} = \mathbf{162} \text{ unique design points}$$

ANSYS additionally re-ran 108 of them after a project update — 270 rows in total. Duplicates differ only at the 5th–6th significant figure (solver micro-noise).

Call: keep all 270 rows

Every ANSYS solve was expensive — throwing them away is wasteful. To avoid train/test leakage, cross-validation is grouped on $(\boldsymbol{m}, \boldsymbol{K}, \boldsymbol{\theta})$ so duplicates always end up in the same fold.

`convert.py` architecture

```
def convert(source=None, out=None):
    src = source if source else _pick_source(prefer_csv=True)
    if src.suffix.lower() == ".csv":
        raw = _read_csv(src)              # skip 7-line preamble
    else:
        raw = _read_xlsx(src)             # drop the "Units" row

    df = _normalize_columns(raw)          # "P1" or "P1 - Force..."
    df = pd.to_numeric(...).dropna()
    df = _convert_units(df)               # MPa -> Pa, mm -> m
    df = _attach_constants(df, ModelConstants())
    df["case_id"] = [f"DP_{i+1:03d}" for i in range(len(df))]
    df.to_csv(out, index=False)
    return df
```

- Two source formats: CSV (preamble + bare `P1`) and XLSX (Units row + `P1 - Force X Component`)
- A single regex `^\s*(P\d+)\b` catches both header forms
- Unit normalisation: MPa→Pa, mm→m
- ANSYS constants (mesh, material) attached row-wise from `ModelConstants`

`preprocess.py`: feature engineering

```
INPUT_FEATURES = ["patient_mass_kg", "K_factor", "force_angle_deg"]
DERIVED_FEATURES = ["force_x_N", "force_z_N", "force_magnitude_N"]

def add_features(df):
    out = df.copy()
    rad = np.deg2rad(out["force_angle_deg"])
    Fmag = out["patient_mass_kg"] * 9.81 * out["K_factor"]
    out["force_x_N"] = Fmag * np.cos(rad)
    out["force_z_N"] = Fmag * np.sin(rad)
    out["force_magnitude_N"] = Fmag
    return out
```

- The model sees three independent inputs: m, K, θ
- $F_x, F_z, |F|$ are derived **analytically** from those three — predict-time and train-time use the same formula, no drift
- F_y and `fixed_support_n_faces` are constant across all rows (zero variance) $\Rightarrow$ dropped from the input vector

CV grouping — `grid_groups()`

```python
GRID_KEYS = ("patient_mass_kg", "K_factor", "force_angle_deg")

def grid_groups(df):
    """Return a group ID per row, equal for ANSYS re-runs."""
    grouped = df[list(GRID_KEYS)].astype(str).agg("|".join, axis=1)
    return grouped.to_numpy()
```

How Recep wires it in `train.py`:

- `GroupKFold(n_splits=5).split(X, y, groups=grid_groups(df))`
- Every duplicate of the same (m, K, θ) triplet shares a fold
- Without this, R^2 would inflate to ≈ 1.0 (leakage)

Bugs I hit on the way

- **The XLSX "Units" row:** the first data row is actually a unit label (`kg, MPa, mm, ...`). pandas inferred `object` dtype for the whole column, breaking arithmetic. Fix: drop the row, then `pd.to_numeric(errors='coerce')`
- **Two header formats:** CSV has `P1`, XLSX has `P1 - Force X Component`. Fix: a single regex $^{*}(P\d+)\_|catchesboth$.
- **`out=OUT_CSV` default argument:** `Python binds defaults at definition time, so monkeypatch in tests was a no-op -- tests silently wrote to the real data/dataset.csv.` Fix: `out=None + resolve inside the function.`

Tests & coverage

File	Statements	Coverage
`src/convert.py`	115	90%
`src/preprocess.py`	27	100%

Key test cases:

- Immutability: `add_features()` does not mutate its input
- Trigonometry: $F_x^2 + F_z^2 = |F|^2$ for every row
- Stale F_x/F_z columns get overwritten (predict-time consistency)
- ANSYS re-runs share a single `grid_groups()` ID

Demo

```
$ ./run.sh convert
wrote 270 rows -> data/dataset.csv
total rows kept: 270 (unique grid points: 162)

$ ./run.sh check | head -6
rows=270  columns=30
unique grid points: 162
non-null target counts:
  safety_factor_neck_min: 270 / 270
  ...
```

The output feeds directly into Recep's `train.py`. The schema is the single source of truth we share with the modelling side.

Questions

Thank you.

Questions?

- `src/convert.py`, `src/preprocess.py`
- `tests/test_preprocess.py` (12 tests, 100% coverage)
- Development log: `docs/PROJECT_NOTES.md` Sprints 1–2